Reviewer Comment:

Reviewer 1

R1R1

The novelty of the core methodology is limited. The main techniques employed in this work (MLP, ONNX conversion, and dynamic/static quantization) are all standard and mature technologies. Therefore, the paper appears more as an engineering optimization study rather than a methodological contribution. Although the reported reduction in inference latency to 64ns seems impressive, several missing evaluations make the practical contribution and significance of the optimization unclear.

Response R1R1

We thank the reviewer for this constructive feedback regarding the novelty and scope of our work. We agree that MLPs, ONNX conversion, and quantization (both static and dynamic) are established and mature technologies in the field of machine learning. However, we respectfully highlight that the primary methodological contribution of this work is not the creation of these individual techniques, but rather the systematic, side-by-side comparative evaluation of their combined integration within a unified reference pipeline. To the best of our knowledge, no existing literature provides such a comprehensive comparative study on a single reference model.

To address the reviewer's concerns regarding the practical significance of our optimization and to demonstrate the generalizability of these techniques, we have significantly expanded our experimental evaluation in the revised manuscript as follows:

- We have extended our optimization and evaluation framework to include two additional, structurally distinct deep learning architectures: a Convolutional Neural Network (CNN) and a Gated Recurrent Unit (GRU). This allows us to assess how well these optimization pipelines scale and perform across different network structures.
- We have conducted an extensive benchmarking study investigating the impact of batch size on both training and inference times. To provide a complete performance profile, these evaluations were performed across both CPU and GPU environments.
- We have compiled these expanded evaluations to compare the trade-offs between model accuracy, hardware utilization, and inference latency (retaining the highly competitive latency of 89.5ns as a benchmark reference, backed by a robust 100-run statistical evaluation to ensure stability).

By documenting these multidimensional comparisons, this study serves as a highly practical deployment guide for researchers and practitioners looking to implement and optimize deep learning models for latency-critical applications.

R1R2

Lack of end-to-end comparison with existing works. The paper only compares the proposed model under different optimization settings (Keras vs. ONNX runtime and different quantization schemes), but does not provide any comparison with existing approaches discussed in Section 2 under similar optimization conditions. In particular, for existing models beyond MLP, it remains unclear whether similar ONNX conversion and quantization pipelines would yield comparable latency improvements. Without such comparisons, it is difficult to determine whether the observed speedup originates from the proposed design itself or simply from standard deployment optimization techniques.

Response R1R2

We appreciate the reviewer's constructive critique regarding the need for an end-to-end comparative analysis with existing works to isolate the source of the observed latency improvements. We agree that comparing our findings against the models discussed in Section 2 under identical conditions would be highly valuable. However, a major challenge in this domain is that the authors of those works have not provided publicly accessible code repositories, pre-trained model weights, or sufficient implementation details to allow for an exact, fair replication on our hardware and compiler stack. Because deep learning latency is highly sensitive to hardware specifications, batch sizes, and minor compiler differences, comparing our optimized results directly against the absolute latency numbers reported in those papers would not be scientifically rigorous or fair.

To overcome this limitation and directly address the reviewer's core question we have implemented and benchmarked two additional deep learning architectures under our exact same deployment pipeline:

- A 1D Convolutional Neural Network (CNN) (representing spatial processing)
- A Gated Recurrent Unit (GRU) network (representing sequential processing)

By training and running these models through our Keras, ONNX, and dynamic/static quantization workflows, we have isolated the effects of the architecture versus the deployment pipeline:

We observe consistent relative speedup trends across all three architectures when converting to ONNX and applying static quantization. This confirms that our optimization pipeline is robust and generalizable.

Despite these optimization-driven speedups, our proposed MLP architecture still achieves the lowest absolute latency 89.5ns whereas the CNN and GRU models stop at significantly higher minimum latencies due to their inherent structural complexities.

This dual-track analysis demonstrates that the ultimate ultra-low latency of 89.5ns is a synergistic result of both the lightweight design of our proposed network and the applied optimization pipeline. We have updated the manuscript to clarify these points and discuss these comparative trends in detail.

R1R3

Existing IDS/DDoS studies have already reported ONNX- and quantization-based compression and acceleration techniques. Therefore, the manuscript should clearly position itself against these works. Specifically, what fundamentally enables the improvement of lower inference latency? Is the gain due to architectural design, model size, hardware configuration, or implementation details? These questions require more detailed experimental analysis and discussion.

Response R1R3

We thank the reviewer for this excellent and highly constructive comment. The question of "what fundamentally enables" the ultra-low latency is critical, and we agree that positioning our work clearly against existing IDS/DDoS literature using these techniques is essential for the paper's impact.

While several outstanding IDS/DDoS studies have indeed investigated ONNX conversion and quantization, they typically report these optimization effects in isolation or tied to a single, specific model configuration. The core contribution of our work is to move beyond "black-box" optimization reports by systematically decoupling and quantifying the individual and combined contributions of:

- By extending our benchmarks to include three structurally diverse architectures (MLP, 1D-CNN, and GRU).
- By comparing performance across CPU and GPU architectures under varying batch sizes.
- By isolating the marginal speedup contributed by Keras-to-ONNX conversion, dynamic quantization, and static quantization.

Based on our expanded experimental analysis, we have updated the manuscript to explicitly answer the reviewer's question.

R1R4

The paper focuses primarily on raw model inference latency. However, for practical real-time deployment, end-to-end latency (e.g., feature extraction, preprocessing, packet parsing, and flow aggregation overhead) is often more important than standalone forward-pass time. Therefore, evaluating only inference time may not sufficiently support the real-time deployment claim. The authors are encouraged to provide an end-to-end evaluation and quantify how much overall system-level latency reduction can actually be achieved under realistic deployment settings.

Response R1R4

We thank the reviewer for this highly practical and insightful comment. The distinction between standalone model inference latency and total end-to-end system latency (encompassing packet parsing, feature extraction, and flow aggregation) is a crucial consideration for real-world, line-rate network deployment.

We agree that preprocessing and feature extraction impose non-trivial latency overheads. However, the core objective of our study is to isolate, benchmark, and minimize the computational bottleneck of the deep learning inference stage itself across different architectures, hardware platforms, and precision formats. In modern high-speed intrusion detection architectures, the upstream network tasks (packet parsing and flow extraction) are increasingly decoupled from the classification engine and offloaded to kernel-level or hardware-accelerated processing pipelines, such as:

- eBPF/XDP (Extended Berkeley Packet Filter / eXpress Data Path): Operating directly in the Linux kernel driver space to parse packets at line rate with sub-microsecond overhead.
- SmartNICs and DPDK (Data Plane Development Kit): Bypassing kernel overheads completely to execute real-time flow aggregation directly in hardware.

To the best of our knowledge, existing IDS/DDoS literature focusing on deep learning optimization traditionally benchmarks standalone forward-pass latency to isolate the compiler and quantization effects from variable network-stack performance.

To address the reviewer's valuable suggestion and clarify the deployment landscape, we have updated the manuscript to:

- Explicitly outline the scope boundaries of our study.
- Formally designate the end-to-end hardware-in-the-loop system evaluation as a key direction for our future work.

We believe this addition significantly enhances the practical value of the paper by providing a realistic deployment blueprint for network security engineers.

R1R5

Minor issue. In Section 3.2, the description "the ratio between the size of the submitted query and the response received" appears to be reversed.

Response R1R5

We thank the reviewer for their exceptionally careful reading of our manuscript and for identifying this error. The reviewer is entirely correct; the description of the ratio was indeed reversed in our original text.

We have corrected this sentence in Section 3.2 of the revised manuscript. The metric is now accurately defined as the ratio of the response payload size to the submitted query payload size. We appreciate this valuable correction, as it ensures the exactness of our methodology's formal description.

Reviewer 2

R2R1

Latency measurement – The reported 64 ns is batch-averaged. For real-time detection, measure single-sample latency (batch size = 1) with proper warm-up and repetitions. Report mean ± SD

Response R2R1

We thank the reviewer for this technically rigorous comment. The distinction between single-sample B = 1 latency and batch-averaged latency is indeed a critical factor when evaluating machine learning deployment paradigms.

While we acknowledge the reviewer's perspective regarding the standard use of B = 1 evaluations in classic real-time edge processing, we respectfully clarify our rationale for focusing on and reporting batch-averaged latencies in this manuscript:

- The primary objective of our work is not to present a standalone, isolated latency record for a single operational state. Instead, we aim to provide a comprehensive, systematic comparison of different compilers (ONNX) and quantization configurations across multiple architectures (MLP, CNN, GRU) on a unified reference platform. Evaluating performance across varying batch configurations is central to mapping these multi-dimensional tradeoffs.
- In high-throughput intrusion detection systems (IDS) and DDoS mitigation pipelines, data does not typically arrive or get processed in a strictly sequential, blocking, single-sample (B = 1) fashion. Modern high-speed packet processing frameworks (such as DPDK or SmartNIC architectures) utilize burst-based processing (e.g., retrieving packets from the NIC ring buffer in bursts/batches of B = 32 or B = 64) to avoid the massive CPU interrupt overhead of packet-by-packet handling. Therefore, profiling batched performance and reporting the amortized, batch-averaged latency provides network security engineers with a far more realistic representation of how these optimized models perform under high-throughput line-rate loads.

We have updated the manuscript to include a dedicated discussion clarifying why burst-based batching modes represent the dominant operational paradigm in target high-throughput deployment architectures, thereby contextualizing our evaluation methodology.

R2R2

Baseline comparisons – Add at least two alternative lightweight detectors (e.g., pruned MLP, 1D-CNN, or TensorRT) to contextualize the speed/accuracy trade-off.

Response R2R2

We thank the reviewer for this constructive recommendation. We agree that contextualizing the performance of our proposed Multilayer Perceptron (MLP) against alternative baseline architectures is essential for demonstrating the relative speed, complexity, and accuracy trade-offs.

To address this valuable suggestion, we have expanded our experimental evaluation to include two alternative, structurally distinct deep learning models running under our exact same deployment and optimization pipeline:

- A 1D Convolutional Neural Network (1D-CNN): Representing a spatial-feature extraction baseline capable of capturing local packet header/payload relationships.
- A Gated Recurrent Unit (GRU) network: Representing a lightweight sequential baseline designed to capture temporal/flow dependencies.

Additionally, to achieve more statistically robust results, we increased the number of iterations in our experiments from 30 (in our previous submission) to 100. By training, evaluating, and running these alternative baselines through our exact same compilation (Keras to ONNX) and quantization (static/dynamic INT8) workflows, we have successfully contextualized both the performance boundaries and the relative speedup trends.

R2R3

Statistical rigor – Provide standard deviations and significance tests for the 30 training runs (Table 4).

Response R2R3

We thank the reviewer for this excellent recommendation. Ensuring that observed differences across different configurations are mathematically sound is crucial for establishing statistical rigor. To address this, we conducted statistical significance testing and our analysis confirmed that the variations in accuracy and F1-score across the different feature subsets are not statistically significant ($p > 0.05$). Because the classification performance remained highly stable and statistically equivalent across these thresholds, we made a conscious design decision to carry out our core inference-time optimizations using the maximum feature set.

R2R4

Data preprocessing – Clarify the source of the extra 110k benign samples. Downsample only the training set to avoid leakage.

Response R2R4

We thank the reviewer for this constructive feedback regarding our data preprocessing workflow, dataset composition, and the prevention of data leakage. We have revised the corresponding paragraph in Section 4 to address these points directly.

To clarify these processes, we provide the following details:

- The target day of the DrDoS attack in the CIC-DDoS-2019 dataset contains an extremely high volume of attack packets but a very scarce number of benign packets. To establish a robust and representative background of normal network behavior, we harvested and merged all available benign traffic profiles from the other days of the same official CIC-DDoS-2019 dataset. This allowed us to compile a total of 113,828 benign samples.
- Even with all 113,828 benign samples consolidated, the ratio of benign to DrDoS attack packets remained heavily imbalanced at approximately 1:44. To prevent biased learning and ensure the models could accurately characterize the minority (benign) class, we applied downsampling.
- Downsampling was applied exclusively to the majority attack class (DrDoS). We preserved 100% of the collected benign samples (113,828 packets) and randomly subsampled the massive DrDoS packet pool down to 227,656 packets, yielding a structured 1:2 (Benign to DrDoS) ratio.
- Because the downsampling process is a simple, uniform random selection of samples from a single, homogeneous class without utilizing any statistical feature parameterization (such as mean, variance, or scaling parameters) or sample duplication it does not transfer any information across the eventual train-test split boundary. Thus, executing this uniform subsampling on the global attack pool prior to the train-test partition is mathematically leak-free and guarantees that our evaluation metrics on the test set remain highly reliable.

We have updated Section 4 of the manuscript to make this preprocessing pipeline and the preservation of the benign traffic explicitly clear to the readers.

R2R5

Reproducibility – Release code, random seeds, and exact library versions.

Response R2R5

We thank the reviewer for emphasizing reproducibility, which is a cornerstone of rigorous scientific and engineering research. We fully agree that providing complete technical specifications is essential for validating and reproducing our experimental results. To address the reviewer's request and guarantee reproducibility, we have taken the following actions:

- We have explicitly documented the exact software dependency versions (such as Python, TensorFlow/Keras, and ONNX Runtime versions), model training hyperparameters, and the specific random seeds within the Supplementary Materials. This ensures that the computational environment and stochastic processes can be replicated.
- We would like to respectfully clarify that we are unable to publicly release the raw source code repository on an open hosting platform (such as GitHub) due to intellectual property constraints and non-disclosure clauses in our active sponsoring/funding agreement. To maintain transparency while respecting these legal agreements, we have integrated a formal Data and Code Availability Statement into the manuscript. Interested researchers may direct inquiries regarding the code and optimization assets to the corresponding author, and access may be granted on a case-by-case basis following review and approval by the sponsoring entity.

R2R6

Acknowledge that latency figures are A100-specific, true single-sample latency may be higher, and robustness was not tested.

Response R2R6

We thank the reviewer for this constructive suggestion. We fully agree that explicitly outlining the hardware-dependency of our benchmarks is vital for providing a balanced, transparent, and scientifically rigorous context for our results.

To address this point, we have updated the Discussion section of the manuscript to formally acknowledge the hardware specificity of our latency profiles. Specifically, our reported absolute latency figures are highly dependent on our specific high-performance execution environment, which leverages an enterprise-grade NVIDIA A100 GPU and high-performance Intel Xeon host CPUs. Deploying these models on lower-specification commodity hardware,

general-purpose processors, or resource-constrained edge platforms will naturally result in different absolute latency metrics.

We appreciate the reviewer's guidance in adding this clarification, which improves the precision and objective presentation of our experimental outcomes.

Editor

ER1

The English is generally understandable, but it requires proofreading and minor corrections before final acceptance. Several typographical errors were identified (e.g., “withouut” instead of “without”, inconsistent spelling of “down sampling” as two words). Additionally, some sentences are overly long or slightly awkward, which occasionally obscures meaning. Abbreviations (e.g., “DrDoS” vs “DNS DrDoS”) should be used consistently throughout the manuscript. A careful copy-edit by a native English speaker or a professional editing service is recommended to improve clarity and readability.

Response ER1

We sincerely thank the reviewers for their careful reading of our manuscript and for pointing out these language and formatting issues.

In response to your valuable feedback, we have undertaken the following actions:

- We have thoroughly proofread the entire manuscript to identify and correct all typographical errors. Specific instances highlighted by the reviewer have been resolved.
- We have standardized key terms and abbreviations across all sections. For instance, "down-sampling" is now spelled consistently with a hyphen, and abbreviations like "DrDoS" and "DNS DrDoS" are applied systematically.
- The entire manuscript has undergone a rigorous revision and copy-editing process by a native English speaker. Overly long or complex sentences have been restructured to enhance flow, precision, and clarity.

We believe these revisions have significantly improved the readability of the manuscript.

ER2

Please add the correlation with the symmetry concept in your abstract and main text so that readers could know clearly the paper's relevance to our journal Symmetry when they start to read the papers.

Response ER2

In accordance with editor’s recommendation, we have revised both the Abstract and the Introduction section to formally introduce and emphasize how the concept of symmetry underlies our proposed framework.

ER3

For commercial funding, the role of the funder must be declared. We recommend inserting the following statements into the manuscript's

Conflict of Interest section:

"The authors declare that this study received funding from XXXXXXX. The funder had the following involvement with the study: XXXXXXX".

"The authors declare that this study received funding from XXXXXXX. The funder was not involved in the study design, collection, analysis, interpretation of data, the writing of this article or the decision to submit it for publication".

Response ER3

We appreciate the editor's guidance regarding the declaration of commercial funding. In accordance with your recommendation and the journal's disclosure policies, we have updated the Conflict-of-Interest section of our manuscript. Because the funding organization played no role in the execution of the research or the preparation of the manuscript, we have adopted the second statement format you provided to explicitly define this relationship.